arXiv:2401.02987v4 [cs.CL] 14 Feb 2024

HAS YOUR PRE-TRAINED MODEL IMPROVED? A MULTI-HEAD POSTERIOR BASED APPROACH

Prince Aboagye[*] Yan Zheng[*†] Junpeng Wang[*†] Uday Singh Saini[*†] Xin Dai[†]
Michael Yeh[†] Yujie Fan[†] Zhongfang Zhuang[†] Shubham Jain[†] Liang Wang[†]
Wei Zhang[*†]

ABSTRACT

The emergence of pre-trained models has significantly impacted Natural Language Processing (NLP) and Computer Vision to relational datasets. Traditionally, these models are assessed through fine-tuned downstream tasks. However, this raises the question of how to evaluate these models more efficiently and effectively. In this study, we explore a novel approach where we leverage the meta-features associated with each entity as a source of worldly knowledge and employ entity representations from the models. We propose using the consistency between these representations and the meta-features as a metric for evaluating pre-trained models. Our method's effectiveness is demonstrated across various domains, including models with relational datasets, large language models, and image models.

1 INTRODUCTION

The rise in user-generated content has led to the widespread adoption of pre-training large models across numerous machine-learning fields. This trend is particularly noticeable in Natural Language Processing (NLP) with GPT (Generative Pretrained Transformer) models (Brown et al., 2020) and vision-language domains with models like CLIP (Radford et al., 2021). Usually, the performance of these models is assessed through various downstream tasks such as Machine Translation (Wang et al., 2023), Factuality (Chen et al., 2019), Question answering (Liang et al., 2023), Multilingual tasks (Bang et al., 2023; Ahuja et al., 2023), which can be relatively expensive if a large number of tasks are considered necessary for a robust evaluation. Is there an alternative approach that offers both efficiency and simplicity for evaluating these models?

Entity representations, also known as embeddings generated from these models, can be utilized directly or indirectly by downstream tasks and fine-tuned as needed. The associated meta-features with these embeddings can be considered as the model's foundational knowledge of the world it's learning from. This could be the class category for image data or semantic and syntactic information for words. Despite having the same meta-features, embeddings differ across models. Therefore, the degree of consistency between the embeddings and meta-features can serve as a performance metric for model evaluation.

Embedding spaces are complex and challenging to interpret or explain. Despite extensive efforts to decipher it, its intricacies go beyond mere linear interpretability, as some research suggests. In this research, we hypothesize that the embeddings reside within a manifold space where Euclidean distance is not an appropriate metric for gauging the similarity between two embeddings. Meta-features are capable of grouping these embeddings into clusters, we assume each forming a sub-manifold space. When the clusters are sufficiently fine-grained, it is possible to approximate each cluster using a Gaussian distribution. Collectively, these clusters form a mixture of Gaussian distributions. By determining the posterior probabilities of the entities, the consistency of meta-features and embeddings can be assessed.

In this study, we introduce a unique approach to evaluate the performance of pre-trained models. Specifically, we:

[*]These authors contributed equally to this work. Correspondence email: wzhan@visa.com
[†]Visa Research, Palo Alto, CA.

1. Adopt a distinct perspective towards the model. Instead of focusing solely on downstream performance, we emphasize the quality of the entities' representations that the model can generate.
2. Consider the features associated with the entity representations as the benchmark to assess their quality. We hypothesize that the meta-features can partition the embedding space into distinct clusters. The quality of these clusters can be evaluated using the posterior probability of Gaussian mixture models.
3. While there are multiple methods to interpret these meta-feature spaces, we present a tree-based approach as an example that uses meta-features to segment entities into clusters.
4. Test our proposed method's effectiveness on various datasets across domains, ranging from recommendation-based to language and image models. We present both qualitative and quantitative evidence to demonstrate the approach's efficacy.

2 Related Work

This section reviews the literature on three areas: 1) Pre-trained models, 2) Vision-Language Pre-training (VLP), and 3) Pretrained Dual-Transformers (PDT) for Bipartite Graphs.

2.1 Pre-trained models

Large Language Models (LLMs): In recent years, significant strides have been made in the realm of Natural Language Processing (NLP), particularly with the advent of the transformer architecture. Attention-based language models such as BERT (Kenton & Toutanova, 2019), GPT (Brown et al., 2020), XLNet (Yang et al., 2019), and T5 (Raffel et al., 2020) have raised the bar in various language benchmarks. Alongside these developments, a plethora of pre-training and fine-tuning algorithms have been devised to enhance the performance of these transformer models. As these models grew in size, the data-driven nature and scaling characteristics of the transformer architecture became evident. These critical findings paved the way for the creation of large language models (LLMs), including LLaMa 2 (Touvron et al., 2023) with 7-70 billion parameters, BLOOM (Workshop et al., 2022) with 176 billion parameters, and GPT4 (OpenAI, 2023) with an astounding 1.7 trillion parameters. These LLMs demonstrate impressive emergent capabilities, such as solving mathematical equations and analyzing articles, competencies not seen in prior smaller language models. These breakthroughs signify the remarkable progress made in this field.

Vision-Language Pre-training (VLP): With the rapid expansion of model capacity and computational resources, the input to deep neural networks has evolved beyond a single modality, such as text or image. Vision-language pre-training (VLP) was introduced to bridge the gap between different modalities, effectively harnessing cross-modality information from both images and text. Leveraging the successful pre-training and fine-tuning paradigm prevalent in NLP, VLP models have demonstrated exceptional performance in complex vision-language tasks. These tasks include image captioning, visual question answering, and visual reasoning. Among the existing studies, a noteworthy contribution is the CLIP (Radford et al., 2021) model, which employs the concept of contrastive learning to align images and text. CLIP simultaneously trains an image encoder and a text encoder on millions of image-text pairs collected from the internet. The resulting encoders have demonstrated impressive performance on downstream tasks due to their zero-shot classification capability.

Pretrained Dual-Transformers (PDT) for Bipartite Graphs: PDT (Dai et al., 2023) focuses on learning contextual knowledge from a user-content interaction dataset, which is depicted as a bipartite graph. The study identifies two key contexts in the graph: user-side and content-side. The goal of learning from these contexts is framed as two contrastive learning tasks and is applied to a recommendation task. Evaluations of two large popular datasets reveal that PDT outperforms baselines in six metrics.

3 ALGORITHM FRAMEWORK

This section presents the algorithmic framework of our proposed metric for evaluating embeddings.

For a given domain, we first collect a large size of entities with rich meta-features. Then for any given pre-trained model, we can generate an embedding dataset denoted as $\mathbf{X} = \mathbf{x}_1, ..., \mathbf{x}_N$, where each $\mathbf{x}_i \in \mathbb{R}^d$ and $1 \leq i \leq N$. Here, N represents the number of entities, and d signifies the dimension of the embeddings. Simultaneously, we can produce a corresponding feature set $\mathbf{F} = \mathbf{f}_1, ..., \mathbf{f}_N$. Each feature $\mathbf{f}_i$ comprises categorical and numerical features. We convert numerical features into categorical ones for consistency. The primary objective is to examine the consistency between these two datasets, $\mathbf{X}$ and $\mathbf{F}$.

In the simplest scenario where $\mathbf{f}_i$ has only one feature, a straightforward segmentation method is to form clusters based solely on these features, with each category creating its cluster. However, when $\mathbf{f}_i$ has m features, and each feature has k_j categories, the number of combinations becomes $\prod_{j=1}^{m} k_j$. This results in a significantly larger number of clusters. We will postpone the discussion on finding the best split to a later session. In the upcoming discussion, we will assume that we already have a split criterion for the dataset $\mathbf{X}$.

4 PROPOSED METHOD: POSTERIOR BASED EMBEDDING EVALUATING METRIC

We aim to evaluate the effectiveness of different models that generate these embeddings to determine the best one. The splitting criteria, $\mathbf{S}$, can divide the entities into a group of clusters $C_1, C_2, ..., C_n$, with each entity belonging to a single cluster, where n is the number of clusters. To evaluate the quality of the cluster, we adopt a posterior-based method.

In the context of GMM, it is assumed that the data is generated from a combination of multiple Gaussian distributions. Each component of this mixture corresponds to one of the distinct clusters within the dataset. The probability of any given data point, $\mathbf{x}$, belonging to the kth cluster is estimated by computing the posterior probability in the GMM framework which can be expressed as follows:

$$P(\theta = k|\mathbf{x}) = \frac{P(\mathbf{x}|\theta = k)P(\theta = k)}{\sum_{j=1}^{m} P(\mathbf{x}|\theta = j)P(\theta = j)}, \tag{1}$$

where $P(\theta = k) = \frac{\text{number of points in the } k\text{th cluster}}{N}$

To assess the quality of the embeddings $\mathbf{X}$ within the context of a splitting $\mathbf{S}$, we compute the overall evaluation metric by averaging the log probabilities of all the embeddings across all clusters. This metric provides an assessment of the quality of the embeddings. We call it the average of the log posterior (ALP).

$$ALP_{\mathbf{S}}^{\mathbf{X}} = \frac{1}{N} \sum_{k=1}^{m} \sum_{\mathbf{x}_i \in C_k} \log P(\theta = k|\mathbf{x}_i) \tag{2}$$

One challenge with the formula above is its high sensitivity to outliers. A single outlier could lead to an extremely large value for $ALP_{\mathbf{S}}^{\mathbf{X}}$. We implement a clipping mechanism for embeddings with very small posterior probabilities to mitigate the impact of such outlier entities. Specifically, if $P(\theta = k|\mathbf{x}_k)$ is less than $k/N * \varepsilon$, we exclude the entity from the $ALP_{\mathbf{S}}^{\mathbf{X}}$ computation.

Another challenge arises when the embeddings exist in large dimensions. If the number of embeddings in each cluster is smaller than the embedding dimension, this leads to a rank-deficient covariance. To address this, we propose a multi-head solution. In this approach, each head is a randomly selected v dimensions from the existing d dimensions and ALP is estimated based on these dimensions. This process is repeated multiple times, and the average results are used which we refer to as the Mean of the . This concept is inspired by the random forest algorithm (Breiman, 2001) and Matryoshka Representation Learning (Kusupati et al., 2022). Additionally, we apply the

routine regularization approach, i.e., adding ϵI to the covariance matrix. The value of ϵ is decided in the following manner.

$$\epsilon = max(\lambda_k/(10D)\lambda_0, 1e{-}8), \tag{3}$$

Where D is the dimensionality of the embeddings and λ_i are the eigenvalues of the covariance matrix (sorted decreasingly by their magnitude). k is the minimum value that satisfies $\frac{\Sigma_{i=0}^{k}\lambda_i}{\Sigma_{i=0}^{D}\lambda_i} > 99.99\%$.

4.1 One Meta Feature Based Clustering

In the simplest scenario, where the feature vector $\mathbf{f}_i$ consists of only one feature, a straightforward and intuitive approach to segmentation is to form clusters based solely on these features. This method is straightforward as it capitalizes on the inherent characteristics of the data. Each unique category within the data forms its distinct cluster, effectively grouping similar items. The consistency of these clusters with the embeddings can serve as a measure of the quality of the embeddings. However, it's important to note that extending this approach to accommodate more than two meta-features is not as straightforward.

4.2 Meta features + representation based segmentation

Inspired by *EmbeddingTree* algorithm, we can construct the tree based on the entities, and all the leaf nodes are the final clusters, specifically: We first convert non-binary categorical features into binary ones by asking yes-no questions regarding each of their categorical values and get the binary feature sets: $\mathbf{G} = \{\mathbf{g}_1, ..., \mathbf{g}_N\}$ ($\mathbf{g}_i \in \{0,1\}^q$, $1 \leq i \leq N$), q denote the total number of converted binary features.

With the processed data, we describe the *EmbeddingTree* algorithm with details in Algorithm 1. We iterate through the q features (line 6) and evaluate them based on the splitting criteria described in Section 4.3 to pick out the best feature for splitting (line 8-10), using the feature's binary value (line 11-13). The above procedure is executed recursively (lines 15-16) until the splitting criterion (Θ), e.g., the number of entities per tree node or the tree depth, is no longer satisfied (line 2). With the given embedding and feature data, the whole procedure is deterministic.

Algorithm 1 Build an *EmbeddingTree*

1: **procedure** BUILDTREE($[\mathbf{X}, \mathbf{F}], q, \Theta$)
2: **if** Θ $is\ not\ satisfied$ **then**
3: return LeafNode($[\mathbf{X}, \mathbf{F}]$)
4: **else**
5: $max_t \leftarrow -\infty$
6: **for** $k \in \{1, ..., q\}$ **do**
7: $t = Embedding - MAP([\mathbf{X}, \mathbf{F}^k])$
8: **if** $t > max_t$ **then**
9: $bestFea = k$
10: $max_t = t$
11: $[\mathbf{X}, \mathbf{F}]_{left} = \{\mathbf{x} \in \mathbf{X} | \mathbf{F}_{bestFea} == 0\}$
12:
13: $[\mathbf{X}, \mathbf{F}]_{right} = \{\mathbf{x} \in \mathbf{X} | \mathbf{F}_{bestFea} == 1\}$
14:
15: Children.Left = $BuildTree([\mathbf{X}, \mathbf{F}]_{left}, q, \Theta)$
16: Children.Right = $BuildTree([\mathbf{X}, \mathbf{F}]_{right}, q, \Theta)$
17: **return** Children

4.3 2-GMM Splitting with Maximum A Posteriori Estimation (MAP)

One critical component of Algorithm 1 is the criterion for selecting the best splitting feature. The criterion is computed based on the approximate MAP for GMMs inspired by (Zheng et al., 2023).

We assume the embedding can be modeled as two mixture Gaussians. The expectation-maximization (EM) algorithm is used to estimate all the parameters and latent variables jointly.

The latent variables, $z_{i,j}$, denote the probability that sample i is in cluster j. With N as the number of observations and J as the number of Gaussian clusters (in this case, $J = 2$), $z = \{z_{1,1}, z_{1,2}, ..., z_{N,J-1}, z_{N,J}\}$, the complete likelihood (including the latent variables) is:

$$P(\mathbf{x}, \mu, \Sigma, w, z) = \prod_{i=1}^{N} \prod_{j=1}^{J} \{w_j \mathcal{N}(\mathbf{x}_n; \mu_j, \Sigma_j^2)\}^{z_{i,j}}, \tag{4}$$

where μ is the mean vectors and Σ is covariance matrix of the Gaussians.

We go through every feature to find the best binary feature that splits the embedding and forms the best GMM. Each candidate binary feature splits the embeddings into two clusters, each formulated as a Gaussian. For each feature, suppose the first s embeddings have feature value $Fk = 0$ and the rest $N - s$ embeddings have feature value $Fk = 1$. We estimate both clusters' weights, means, and variances using the maximum likelihood estimation (MLE).

$$\hat{\mu_1} = \frac{1}{s} \sum_{i=1}^{s} \mathbf{x_i}, \quad \hat{\Sigma_1} = \frac{1}{s} \sum_{i=1}^{s} (\mathbf{x_i} - \hat{\mu}_\mathbf{1})(\mathbf{x_i} - \hat{\mu}_\mathbf{1})^T, \quad \hat{w_1} = \frac{s}{N},$$

$$\hat{\mu_2} = \frac{1}{N-s} \sum_{i=s+1}^{N} \mathbf{x_i}, \quad \hat{\Sigma_2} = \frac{1}{N-s} \sum_{i=s+1}^{N} (\mathbf{x_i} - \hat{\mu}_\mathbf{2})(\mathbf{x_i} - \hat{\mu}_\mathbf{2})^T,$$

$$\hat{w_2} = \frac{N-s}{N}.$$

In other words, our algorithm performs a hard clustering rather than the soft clustering of GMM. Thus, if $\mathbf{x}_i$ is in cluster j, then $z_{i,j} = 1$ and $z_{i,j'} = 0$ for all $j \neq j'$. Given this approximation, the likelihood can be obtained by summing over the z:

$$P(\mathbf{x}, \mu, \Sigma, w) = \sum_{z} \prod_{i=1}^{N} \prod_{j=1}^{J} \{w_j \mathcal{N}(\mathbf{x}_n; \mu_j, \Sigma_j^2)\}^{z_{i,j}} \tag{5}$$

Note that $z_{(i \in (0,s], j=1)} = z_{(i \in [s+1,N), j=2)} = 1$ and $z_{i,j} = 0$, otherwise, the above equation can be simplified to:

$$P(\mathbf{x}, \mu, \Sigma, w) = \prod_{i=1}^{s} w_1 \mathcal{N}(\mathbf{x}_i; \mu_1, {\Sigma_1}^2) \prod_{i=s+1}^{N} w_2 \mathcal{N}(\mathbf{x}_i; \mu_2, {\Sigma_2}^2). \tag{6}$$

We can treat each split feature as another random variable θ. To choose the best-split feature, we want to maximize $P(\mathbf{x}, \mu, \Sigma, w, \theta)$; in other words, we want to find θ that gives the largest $P(\mathbf{x}, \mu, \Sigma, w)$.

4.4 Finding the Best Splitting Point

For simplicity, we only consider θ as the random variable we want to estimate; by injecting the prior information into the formula, we can treat each splitting feature with a different weight. By applying Maximum A Posteriori Estimation (MAP), we can formulate the problem as follows:

$$P(\theta_i | \mathbf{x}) = \frac{P(\mathbf{x}|\theta_i) P(\theta_i)}{\sum_{j=1}^{q} P(\mathbf{x}|\theta_j) P(\theta_j)}, \tag{7}$$

where q is the number of possible splits.

By plugging (3) into (4), we can get

$$P(\theta_i | \mathbf{x}) = \frac{\prod_{k=1}^{s} w_1 \mathcal{N}(\mathbf{x}_k; \mu_1, {\Sigma_1}^2, \theta_i) \prod_{k=s+1}^{N} w_2 \mathcal{N}(\mathbf{x}_k; \mu_2, {\Sigma_2}^2, \theta_i) p(\theta_i)}{\sum_{j=1}^{q} \prod_{k=1}^{s} w_1 \mathcal{N}(\mathbf{x}_k; \mu_1, {\Sigma_1}^2, \theta_j) \prod_{k=s+1}^{N} w_2 \mathcal{N}(\mathbf{x}_k; \mu_2, {\Sigma_2}^2, \theta_j) p(\theta_j)}. \tag{8}$$

Plugging in the estimates for all the parameters and taking the log of $P(\theta_i|\mathbf{x})$, we can get

$$\log \hat{P}(\theta_i|\mathbf{x}) = \sum_{i=1}^{s}[\log \hat{w}_1 + \log \mathcal{N}(\mathbf{x}_i; \hat{\mu}_1, \hat{\Sigma_1}^2)] + \sum_{i=s+1}^{N}[\log \hat{w}_2 + \log \mathcal{N}(\mathbf{x}_i; \hat{\mu}_2, \hat{\Sigma_2}^2)] + \log p(\theta_i)$$
$$- \log(\sum_{j=1}^{q}\prod_{k=1}^{s} w_1 \mathcal{N}(\mathbf{x}_k; \hat{\mu}_1, \hat{\Sigma}_1^2, \theta_j) \prod_{k=s+1}^{N} w_2 \mathcal{N}(\mathbf{x}_k; \hat{\mu}_2, \hat{\Sigma}_2^2, \theta_j) p(\theta_j)). \quad (9)$$

By applying this formula, we can use the prior knowledge of the importance of the feature to find the split that maximizes $\log \hat{P}$.

4.5 Embedding Comparison based on the same splitting criteria

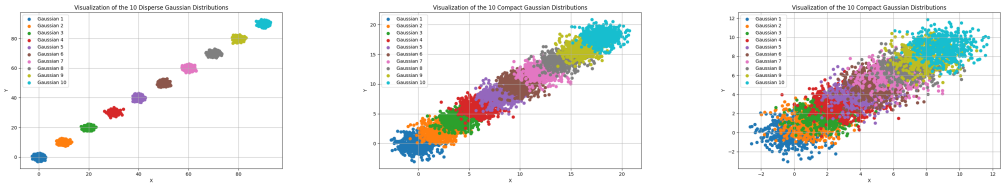
If we have two sets of embeddings, $\mathbf{X_A} = \{\mathbf{x_A}_1, \ldots, \mathbf{x_A}_N\}$, and $\mathbf{X_B} = \{\mathbf{x_B}_1, \ldots, \mathbf{x_B}_N\}$, both trained on the same dataset but using different models, denoted as models A and B, where $(\mathbf{x_A}_i, \mathbf{x_B}_i \in \mathbb{R}^p, 1 \leq i \leq N)$, we can generate two corresponding splitting criteria, $\mathbf{S_A}$ and $\mathbf{S_B}$. The objective now is to assess and compare the quality of these two sets of embeddings. Let's represent $ALP_{\mathbf{S^A}}^{\mathbf{X^A}}$ as $ALP_{\mathbf{A}}^{\mathbf{A}}$ for embeddings $\mathbf{X_A}$ and splitting criteria $\mathbf{S_A}$. Given two sets of embeddings, $\mathbf{X_A}$ and $\mathbf{X_B}$, along with two corresponding splitting criteria, $\mathbf{S_A}$ and $\mathbf{S_B}$, we can define four metrics: $ALP_{\mathbf{A}}^{\mathbf{A}}$, $ALP_{\mathbf{B}}^{\mathbf{B}}$, $ALP_{\mathbf{A}}^{\mathbf{B}}$, and $ALP_{\mathbf{B}}^{\mathbf{A}}$. We need to fix the splitting criteria to do clustering, so a proper comparison should be between $ALP_{\mathbf{A}}^{\mathbf{A}}$ and $ALP_{\mathbf{A}}^{\mathbf{B}}$ or between $ALP_{\mathbf{B}}^{\mathbf{A}}$ and $ALP_{\mathbf{B}}^{\mathbf{B}}$.

5 Experimental Analysis

In this experimental session, we initially conducted experiments on a synthetic dataset to verify the effectiveness of our proposed algorithm. Following this, we further evaluate the results in three areas: the MovieLens dataset (Harper & Konstan, 2015) for relational models, spatial datasets (Gurnee & Tegmark, 2023) for large language models, and the Robustness library (Engstrom et al., 2019a;b; Santurkar et al., 2019; 2020) for image models.

5.1 Synthetic Dataset: Gaussian Mixture Model (GMM) of Ten Gaussian Distributions

To validate the efficacy of our proposed posterior-based embedding evaluation metric, as outlined in Equation 2, we designed an experiment encompassing three scenarios, each with 10 clusters. These clusters were generated such that they are either perfectly separated, partially overlapping, or perfectly overlapping and are all generated using a Gaussian Mixture Model. Figure 1 presents the results from these scenarios. As anticipated, the Average of the Log Posterior (ALP) scores for the ten (10) perfectly separated Gaussian Distributions was 0, and the accuracy of the clusters assigned from the posterior matrix was 100%. In the case of 10 partially overlapping Gaussian Distributions, the ALP score was -0.3285, and the accuracy of the clusters assigned from the posterior matrix was 86.96%. Similarly, for the ten (10) perfectly overlapping Gaussian Distributions, the ALP score was -0.9372, and the accuracy of cluster assignment was 57.34%.



(a) Perfectly separated clusters (b) Partially overlapping clusters (c) Perfectly overlapping clusters

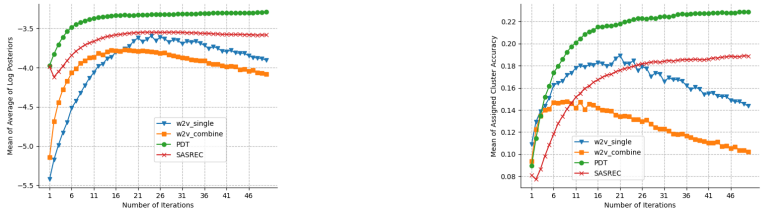
Figure 1: Illustration on a 2D synthetic dataset consisting of 10 Gaussian distributions that are perfectly separated, partially overlapping, and perfectly overlapping.

5.2 MOIVE LENS DATASET FOR RELATIONAL

MovieLens-25M consists of 25,000,095 reviews made by 162,541 reviewers on 59,047 movies. We compare the following models to generate the movie embeddings. Word2vec (Mikolov et al., 2013), PDT (Dai et al., 2023) and SASRec (Kang & McAuley, 2018). From the Word2Vec model, we generate two distinct types of embedding representations, specifically `w2v_single` and `w2v_combine`. In the case of `w2v_single`, we create a sequence of movies for each reviewer's review, sort it based on time, and then employ the Word2vec model to learn the movie embeddings. On the other hand, `w2v_combine` not only includes the sequences of movies for reviewers but also incorporates sequences of reviewers for each movie and reviewer/movie pairs as part of the training data. For both `SASRec` and `PDT`, we generate sequences of nine movies for each reviewer. Additionally, for `PDT`, we generate sequences of reviewers for each movie, as this is a necessary input for `PDT`. `SASRec` is trained using BPR loss, while `PDT` is trained using two contrastive losses. Both `PDT` and `SASRec` are contextualized embeddings, while `w2v_single` and `w2v_combine` are static embeddings. We employ two clustering techniques. The first approach involves clustering by single meta-features, such as year and genre. We also apply the Embedding tree-based method to generate the tree for both year and genre features and use the leaf nodes as clusters.

5.2.1 MOVIE LENS DATASET: CLUSTERING BY YEAR

We evaluated and compared four kinds of embedding representations trained on the MovieLens Dataset. These were trained at various iteration levels; we use the "year" feature as labels to cluster the embeddings. As illustrated Figure 2 (a), the `PDT` and `SRSREC` embedding performed better than all embeddings across all iterations, as seen in the Mean of the average of log posteriors plot. During the early stages of training from iteration 1 to 16, `w2v_combine` outperformed `w2v_single`. However, in the later stages from iteration 16 to 50, `w2v_single` superseded `w2v_combine`.



(a) Mean of the average of log posteriors

(b) Mean of accuracy of assigned clusters

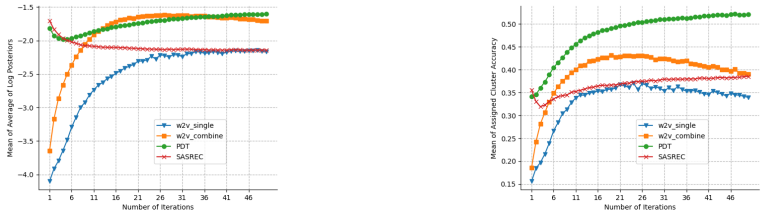
Figure 2: Mean of the average of the log posterior and accuracy on the MovieLens dataset by clustering on year.

As depicted in the Mean of accuracy of assigned clusters plot of Figure 2 (b), `PDT` and `SRSREC` demonstrates a consistent and stable performance over all other types of embeddings across all iterations. Generally, `w2v_single` exceeds the performance of `w2v_combine`. This suggests that contextualized embeddings, specifically `PDT` and `SRSREC`, most effectively encode year information and remain stable across all iterations. Also, `w2v_single` demonstrates superior encoding of year information compared to `w2v_combine`.

5.2.2 MOVIE LENS DATASET: CLUSTERING BY GENRE

Here, we created clusters with the genre features as labels. We then compute and report the Mean of the average of log posteriors and the Mean of accuracy of assigned clusters. These findings are presented in Figure 3. Contrary to the consistent pattern observed with year features as labels, the genre features do not exhibit a similar consistency. From Figure 3 (a), it's noticeable that the `PDT` embedding generally outperforms both `SASRec` and `w2v_single` over all iterations. Furthermore, `SASRec` surpasses `w2v_single` from the 1st to the 40th iteration, after which they both plateau with similar scores. Between the 12th and 36th iterations, `w2v_combine` is observed to outperform `PDT`. Moving to the Mean accuracy of the assigned clusters plot (Figure 3 (b)), it's evident that `PDT` consistently outperforms all other embedding types across all iterations. Generally, `w2v_combine`

surpasses both SASRec and w2v_single, except for the first and third iterations where SASRec exceeds w2v_combine.



(a) Mean of the average of log posteriors

(b) Mean of accuracy of assigned clusters

Figure 3: Mean of the average of the log posterior and accuracy on the MovieLens dataset by clustering on the genre.

5.2.3 Movie Lens Dataset: Clustering with Tree Leaf Nodes on Genre and Year as the Meta Features

We also explored the case where we used the tree leaf nodes from the embedding tree constructed with year and genre as meta-features. The Mean average of log posteriors of assigned clusters is calculated and reported, as shown in Figure 4 (a). The PDT and SASRec embeddings consistently surpass other embeddings throughout all iterations. However, we notice that w2v_combine surpasses w2v_single from the 1st to the 26th iteration, but w2v_single overtakes w2v_combine from the 26th to the 50th iteration. The Mean accuracy of assigned clusters, illustrated in Figure 4 (b), clearly shows that the PDT and SASRec embedding exhibit a steady and consistent increase in performance compared to all other embeddings across all iterations. This is followed by w2v_single, which generally surpasses w2v_combine.

(a) Mean of the average of log posteriors

(b) Mean of accuracy of assigned clusters

Figure 4: Mean of the average of the log posterior and accuracy on the MovieLens dataset by clustering with tree leaf nodes.

All the above experiment results suggest that the contextualized embeddings SASRec and PDT are more effective at capturing the semantic and structural relationships in the input data compared to their static embedding counterparts, namely w2v_single and w2v_combine, which meet our expectations.

5.3 Experiments with Embedding from Llama-2 Models

In this session, we implement the proposed concept in the field of large language models and use Llama-2 as an example.

Models: The released Llama-2 models (Touvron et al. (2023)) have three versions with different model sizes and embedding dimensionalities. These details have been included in Table 1.

Datasets: We use the Llama-2 models to generate embeddings for three spatial datasets introduced by (Gurnee & Tegmark, 2023). The details of these datasets are shown in Table 2. Each dataset

instance is a word/phrase for a location (e.g., the name of a city, a university, or a place of interest). Based on the spatial location as a meta feature, we generate the clusters for each dataset, i.e., based on the continent, state, and borough of the instances from the *World Place*, *USA Place*, and *NYC Place* dataset, respectively.

Table 1: Details of the Llama-2 models.

Model	#head	#layer	#dim
Llama-2-7b	32	32	4096
Llama-2-13b	40	40	5120
Llama-2-70b	64	80	8192

Table 2: Details of the three spatial datasets.

Dataset	#clusters	#samples	Cluster
World Place	8	39585	continent
US Place	49	29997	state
NYC Place	7	19838	borough

Embedding Quality: We feed the three datasets into the three (3) Llama-2 models and get their activations/embeddings at each layer of the models. Specifically, we obtain 32, 40, and 80 sets of embeddings for the three sets of models (as the three models have 32, 40, and 80 layers, respectively). We use our proposed method for each set of embeddings to compute the posterior of individual instances falling into the correct cluster.

From Figure 5 and Figure 6, we present the effectiveness of embeddings across three different datasets and models. The x-axis of each graph indicates the percentage of layers relevant to the corresponding Llama-2 models, while the y-axis represents the Mean average of log posteriors and accuracy. The results show a noticeable upward trend in the quality of embeddings from the initial to the final layers. Specifically, in the World_Place, USA_Place, and NYC_Place datasets, the green lines, which denote the larger Llama-2-70b model, exhibit the highest levels of posteriors and accuracy. This indicates that the Llama-2-70b model is more adept at incorporating location data compared to the other models.

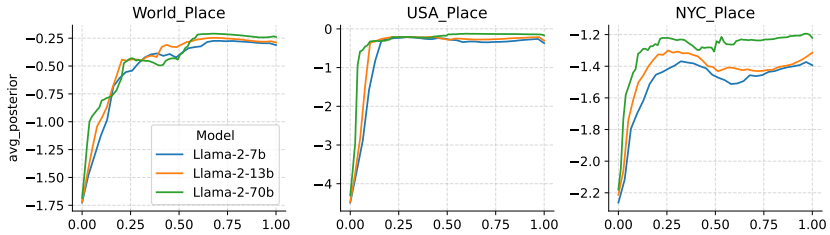


Figure 5: Embedding quality over model layers (average of the log posterior). The dimensions are divided into subsets, each comprising 128 dimensions.

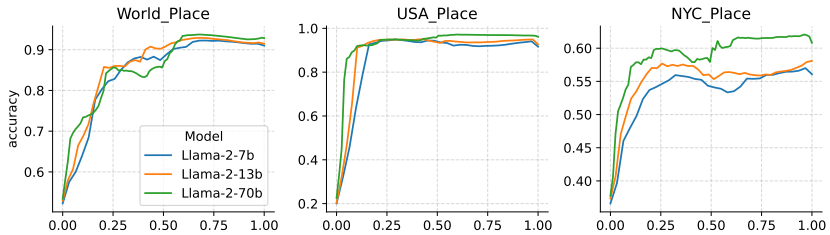


Figure 6: Embedding quality over model layers (accuracy of assigned clusters). The dimensions are divided into subsets, each comprising 128 dimensions.

5.4 Experiment analysis of CLIP Embeddings on the BREEDS hierarchy

In this section, we assess the ability of our metric to predict the classification performance on embeddings of 3 pre-trained CLIP (Radford et al., 2021) models on datasets from the Breeds Hierarchy

(Santurkar et al., 2020), namely datasets entity 13, entity 30, living 17 and nonliving 26. Next, we briefly describe the Breeds Dataset and the CLIP models used for our evaluation.

Dataset: Breeds Hierarchy (Santurkar et al., 2020) was created by leveraging and modifying the WordNet (Miller, 1994) hierarchy for the ImageNet (Deng et al., 2009) dataset to group together semantically similar classes into one (1) superclass. The original purpose of creating this hierarchy was to use the subpopulations present in superclasses to detect a model's robustness to distribution shift. For this experiment, we leverage the entire dataset to evaluate the performance of our metric in predicting the generalization of CLIP models on those embeddings.

Models: For this study, we use CLIP-ViT transformers ViT-L/14, ViT-B/32, and ViT-B/16 trained on 224 px $\times$ 224 px images as image encoders, where ViT-L/14 encodes each image into a 768-dimensional embedding and ViT-B/16 and ViT-B/32 encode each image as a 512-dimensional embedding. We obtained model embeddings for each network from the final image encoder and the mean embeddings of internal multi-head self-attention layers. We train a linear probe on the ImageNet (Deng et al., 2009) training subset of the Breeds dataset and learn the parameters for estimating the log posterior (ALP). To compute the average log posterior, we don't split the embeddings into multiple blocks; therefore, the average log posterior (ALP) demonstrates the flexibility of our approach. We correlate the performance of these two metrics to understand our trained posterior's behavior better and present the analysis and results next.

Table 3: Pearson's Correlation on Breeds Datasets (Part 1): Comparing layerwise log posterior and linear probe accuracy across regularization levels in the training and validation sets of the Breeds Hierarchy.

Entity 13 Dataset

	Train Set			Val. Set		
Reg.	B16	B32	L14	B16	B32	L14
Diag	.99	.99	.98	.99	.99	.97
10^{-3}	.97	.99	.97	.97	.98	.95
10^{-6}	.97	.96	.98	.99	.98	.99
10^{-9}	.96	.95	.98	.99	.99	.99

Entity 30 Dataset

	Train Set			Val. Set		
Reg.	B16	B32	L14	B16	B32	L14
Diag	.99	.99	.99	.99	.99	.98
10^{-3}	.98	.97	.98	.98	.98	.98
10^{-6}	.85	.83	.89	.97	.96	.99
10^{-9}	.8	.76	.87	.96	.96	.98

Table 4: Pearson's Correlation on Breeds Datasets (Part 2): Comparing layerwise log posterior and linear probe accuracy across regularization levels in the training and validation sets of the Breeds Hierarchy.

Living 17 Dataset

	Train Set			Val. Set		
Reg.	B16	B32	L14	B16	B32	L14
Diag	.99	.99	.99	.99	.99	.98
10^{-3}	.93	.93	.95	.97	.96	.99
10^{-6}	.68	.65	.72	.93	.94	.97
10^{-9}	.57	.49	.64	.93	.95	.97

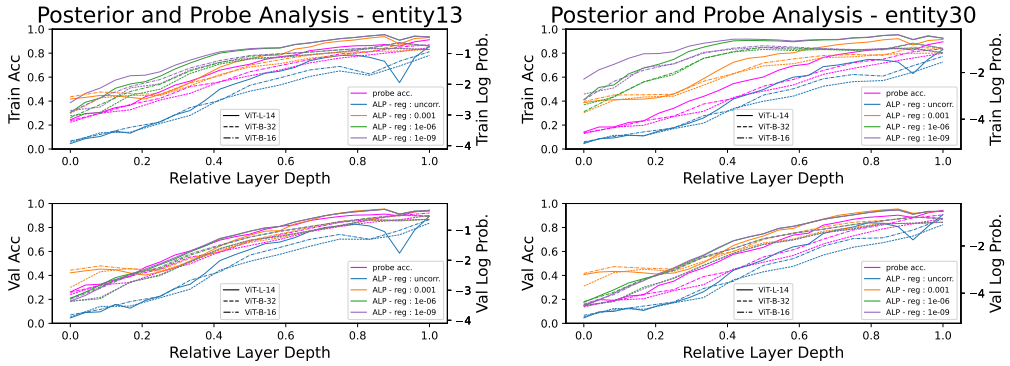
Non Living 26 Dataset

	Train Set			Val. Set		
Reg.	B16	B32	L14	B16	B32	L14
Diag	.99	.99	.98	.99	.99	.98
10^{-3}	.98	.95	.98	.98	.98	.97
10^{-6}	.72	.67	.75	.97	.97	.99
10^{-9}	.54	.42	.64	.96	.97	.98

For this experiment, we measured the correlation between average log posteriors and linear probe accuracy learned and computed over the Breeds training and validation set embeddings. The results are shown in Table 3 and Table 4 for respective datasets from Figure 7 and Figure 8. Based on those results, we demonstrate that average log posterior and linear probe performance correlates strongly across the depth of the network when measured via Pearson's correlation. This is across various settings of regularizations (both with Independence assumptions and Tikhonov[1] Regularization) of the class-wise covariance matrices for our learned average log posterior metric for various Breeds Datasets and CLIP Models.

Our results with a layerwise analysis of Pre-trained CLIP Models comparing our metric with a linear probe on internal activations help us assert that the log posterior is predictive of an embedding's

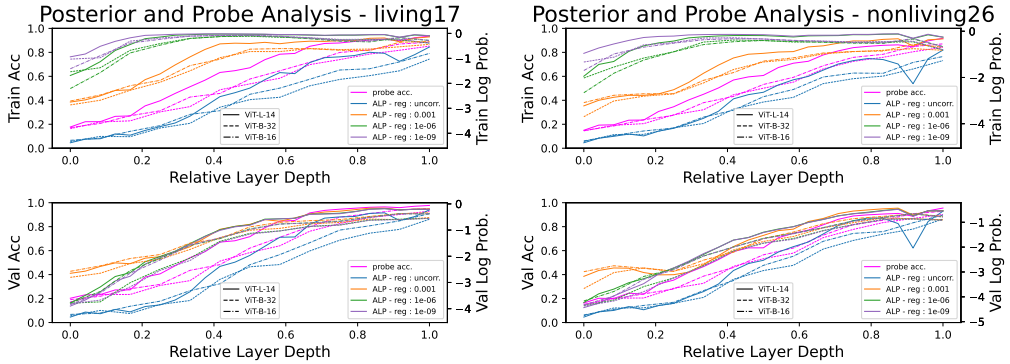
[1]https://web.eecs.umich.edu/ aey/recent/regular.pdf



(a) CLIP Model on entity13.

(b) CLIP Model on entity30.

Figure 7: Evolution of layerwise log posterior and linear probe accuracies for CLIP Models across varying regularization strengths, demonstrating correlations between log posterior and linear probe performance across the depth of various CLIP Models. Quantitative results are shown in Table 3 and Table 4.



(a) CLIP Models on living17.

(b) CLIP Models on nonliving26.

Figure 8: Evolution of layerwise log posterior and linear probe accuracies for CLIP Models across varying regularization strengths, demonstrating correlations between log posterior and linear probe performance across the depth of various CLIP Models. A more detailed breakdown of the results comparing the correlations on each dataset for various settings of regularizations corresponding to Figure 7 and Figure 8 is shown in Figure 9 - Figure 16 of Appendix A.1.

downstream classification performance, even if a set of data that was used to generate the embedding wasn't part of the models training. With Pearson correlations greater than 0.9 for a majority of settings in Table 3 and Table 4, we can confidently establish its value as a metric that can distinguish between good and bad-performing embeddings generated by various models. Additionally we also show the corresponding spearman correlations in Table 5 and Table 6.

Table 5: Spearman's Correlation on Entity Datasets

Entity 13 Dataset

	Train Set			Val. Set		
Reg.	B16	B32	L14	B16	B32	L14
Diag	.98	.99	.95	.99	.99	.94
10^{-3}	.97	.99	.98	.97	.98	.96
10^{-6}	.99	.97	.99	.99	.99	.99
10^{-9}	.99	.98	.98	.99	.99	.99

Entity 30 Dataset

	Train Set			Val. Set		
Reg.	B16	B32	L14	B16	B32	L14
Diag	.99	.99	.98	1.00	.99	.98
10^{-3}	.98	1.0	.99	.97	.98	.96
10^{-6}	.82	.79	.96	.99	1.00	.99
10^{-9}	.72	.67	.91	.99	1.00	.99

Table 6: Spearman's Correlation on Living and Non Living Datasets

Living 17 Dataset

	Train Set			Val. Set		
Reg.	B16	B32	L14	B16	B32	L14
$Diag$	1.00	.99	.99	1.00	.99	.98
10^{-3}	.9	.97	.98	1.00	.99	.99
10^{-6}	.47	.41	.42	.99	1.00	.99
10^{-9}	.3	.16	.26	1.00	1.00	.99

Non Living 26 Dataset

	Train Set			Val. Set		
Reg.	B16	B32	L14	B16	B32	L14
$Diag$	.99	1.00	.97	1.00	1.00	.97
10^{-3}	.99	.99	.98	.97	.97	.95
10^{-6}	.62	.48	.77	.99	.98	.99
10^{-9}	.46	.26	.61	.99	.98	.99

6 CONCLUSION

This work introduces a novel method for evaluating pre-trained models. Instead of using costly and time-consuming fine-tuned downstream tasks for evaluation, we propose using the consistency between entity embeddings and their associated meta-features as a performance metric. Our method has been effectively tested across various domains and datasets in relational datasets, Natural Language Processing, and Computer Vision, providing a more efficient and equally rigorous alternative for pre-trained model evaluation.

REFERENCES

Kabir Ahuja, Harshita Diddee, Rishav Hada, Millicent Ochieng, Krithika Ramesh, Prachi Jain, Akshay Nambi, Tanuja Ganu, Sameer Segal, Mohamed Ahmed, Kalika Bali, and Sunayana Sitaram. MEGA: Multilingual evaluation of generative AI. In *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL https://openreview.net/forum?id=jmopGajkFY.

Yejin Bang, Samuel Cahyawijaya, Nayeon Lee, Wenliang Dai, Dan Su, Bryan Wilie, Holy Lovenia, Ziwei Ji, Tiezheng Yu, Willy Chung, Quyet V. Do, Yan Xu, and Pascale Fung. A multitask, multilingual, multimodal evaluation of ChatGPT on reasoning, hallucination, and interactivity. In Jong C. Park, Yuki Arase, Baotian Hu, Wei Lu, Derry Wijaya, Ayu Purwarianti, and Adila Alfa Krisnadhi (eds.), *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 675–718, Nusa Dua, Bali, November 2023. Association for Computational Linguistics. URL https://aclanthology.org/2023.ijcnlp-main.45.

Leo Breiman. Random forests. *Machine learning*, 45:5–32, 2001.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Anthony Chen, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. Evaluating question answering evaluation. In Adam Fisch, Alon Talmor, Robin Jia, Minjoon Seo, Eunsol Choi, and Danqi Chen (eds.), *Proceedings of the 2nd Workshop on Machine Reading for Question Answering*, pp. 119–124, Hong Kong, China, November 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-5817. URL https://aclanthology.org/D19-5817.

Xin Dai, Yujie Fan, Zhongfang Zhuang, Shubham Jain, Chin-Chia Michael Yeh, Junpeng Wang, Liang Wang, Yan Zheng, and Wei Zhang. Pdt: Pretrained dual transformers for time-aware bipartite graphs. *arXiv preprint arXiv:2306.01913*, 2023.

Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 248–255, 2009. doi: 10.1109/CVPR.2009.5206848.

Logan Engstrom, Andrew Ilyas, Shibani Santurkar, and Dimitris Tsipras. Robustness (python library), 2019a. URL https://github.com/MadryLab/robustness.

Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Brandon Tran, and Aleksander Madry. Adversarial robustness as a prior for learned representations, 2019b.

Wes Gurnee and Max Tegmark. Language models represent space and time. *arXiv preprint arXiv:2310.02207*, 2023.

F Maxwell Harper and Joseph A Konstan. The movielens datasets: History and context. *Acm transactions on interactive intelligent systems (tiis)*, 5(4):1–19, 2015.

Wang-Cheng Kang and Julian McAuley. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*, pp. 197–206. IEEE, 2018.

Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of naacL-HLT*, volume 1, pp. 2, 2019.

Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham M. Kakade, Prateek Jain, and Ali Farhadi. Matryoshka representation learning. In *Neural Information Processing Systems*, 2022. URL https://api.semanticscholar.org/CorpusID:252683450.

Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bobby Yan, Ce Zhang, Christian Alexander Cosgrove, Christopher D Manning, Christopher Re, Diana Acosta-Navas, Drew Arad Hudson, Eric Zelikman, Esin Durmus, Faisal Ladhak, Frieda Rong, Hongyu Ren, Huaxiu Yao, Jue WANG, Keshav Santhanam, Laurel Orr, Lucia Zheng, Mert Yuksekgonul, Mirac Suzgun, Nathan Kim, Neel Guha, Niladri S. Chatterji, Omar Khattab, Peter Henderson, Qian Huang, Ryan Andrew Chi, Sang Michael Xie, Shibani Santurkar, Surya Ganguli, Tatsunori Hashimoto, Thomas Icard, Tianyi Zhang, Vishrav Chaudhary, William Wang, Xuechen Li, Yifan Mai, Yuhui Zhang, and Yuta Koreeda. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. URL https://openreview.net/forum?id=iO4LZibEqW. Featured Certification, Expert Certification.

Tomas Mikolov, Kai Chen, Gregory S. Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *International Conference on Learning Representations*, 2013. URL https://api.semanticscholar.org/CorpusID:5959482.

George A. Miller. WordNet: A lexical database for English. In *Human Language Technology: Proceedings of a Workshop held at Plainsboro, New Jersey, March 8-11, 1994*, 1994. URL https://aclanthology.org/H94-1111.

OpenAI. Gpt-4 technical report, 2023.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision, 2021.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020.

Shibani Santurkar, Dimitris Tsipras, Brandon Tran, Andrew Ilyas, Logan Engstrom, and Aleksander Madry. Image synthesis with a single (robust) classifier, 2019.

Shibani Santurkar, Dimitris Tsipras, and Aleksander Madry. Breeds: Benchmarks for subpopulation shift, 2020.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

Longyue Wang, Chenyang Lyu, Tianbo Ji, Zhirui Zhang, Dian Yu, Shuming Shi, and Zhaopeng Tu. Document-level machine translation with large language models. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 16646–16661, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.1036. URL https://aclanthology.org/2023.emnlp-main.1036.

BigScience Workshop, Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, et al. Bloom: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100*, 2022.

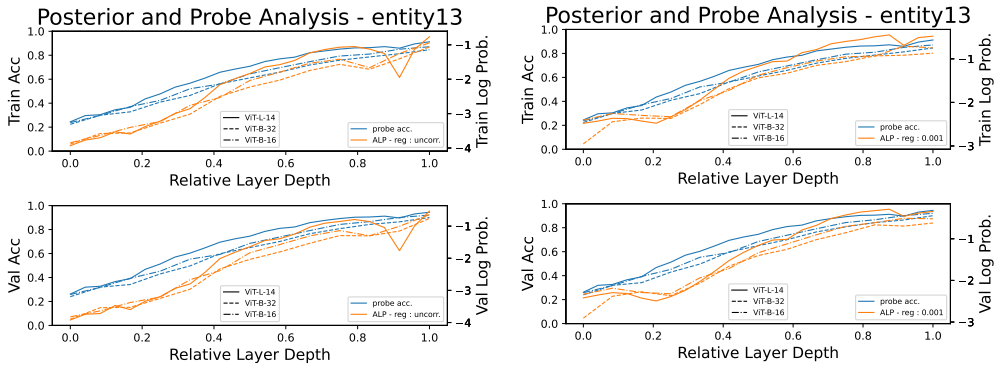
Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Russ R Salakhutdinov, and Quoc V Le. Xlnet: Generalized autoregressive pretraining for language understanding. *Advances in neural information processing systems*, 32, 2019.

Yan Zheng, Junpeng Wang, Chin-Chia Michael Yeh, Yujie Fan, Huiyuan Chen, Liang Wang, and Wei Zhang. Embeddingtree: Hierarchical exploration of entity features in embedding. In *2023 IEEE 16th Pacific Visualization Symposium (PacificVis)*, pp. 217–221. IEEE, 2023.

A Appendix

A.1 Fixed regularization cross Model analysis of CLIP Embeddings on the BREEDS hierarchy

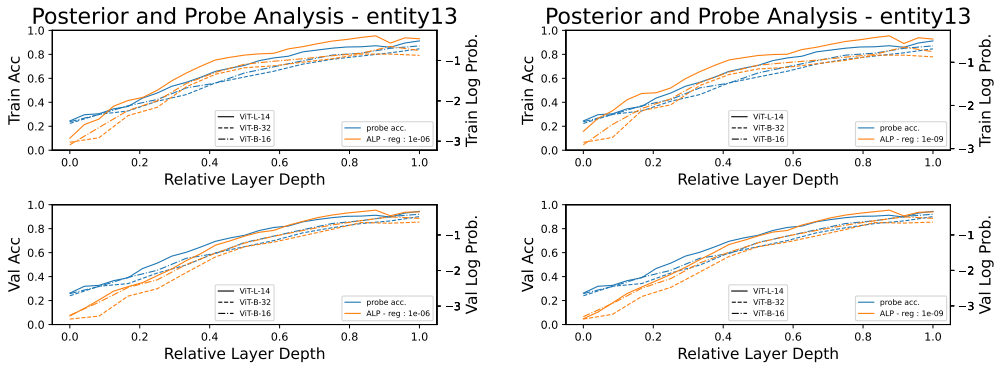
In this section, we break the constituents of Figure 7a, Figure 8, Table 3 and Table 4 into individual plots comparing the behavior of ALP and linear probe accuracy for each CLIP model on a given dataset for varying regularization schemes. A complete per regularization breakdown for the 3 CLIP Models corresponding to entity-13 from Figure 7a is provided in Figure 9 and Figure 10, for entity-30 in Figure 7b, the same is shown in Figure 11 and Figure 12. For living-17 and non-living-26 from Figure 8a and Figure 8b, the analysis is shown in Figure 13, Figure 14 and Figure 15, Figure 16 respectively.



(a) Regularization - Uncorr.

(b) Regularization 10^{-3}.

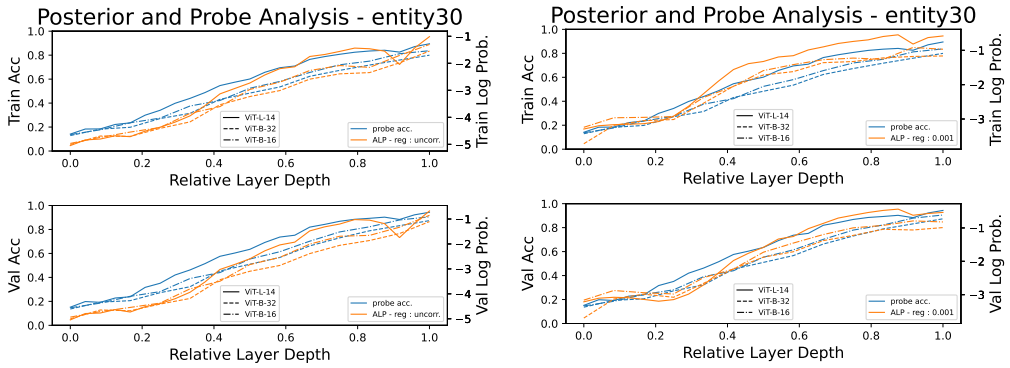
Figure 9: Regularization-wise analysis of layerwise ALP vs. linear probe performance for entity 13 breeds dataset.



(a) Regularization 10^{-6}.

(b) Regularization 10^{-9}.

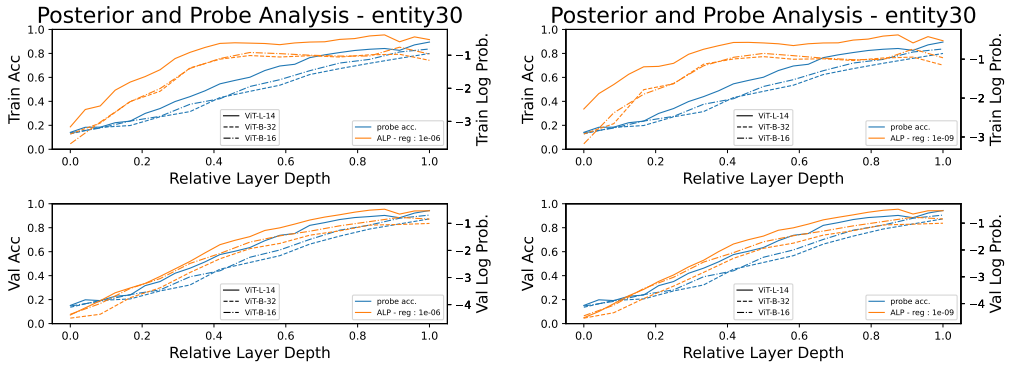
Figure 10: Regularization-wise analysis of layerwise ALP vs. linear probe performance for entity 13 breeds dataset.



(a) Regularization - Uncorr.

(b) Regularization 10^{-3}.

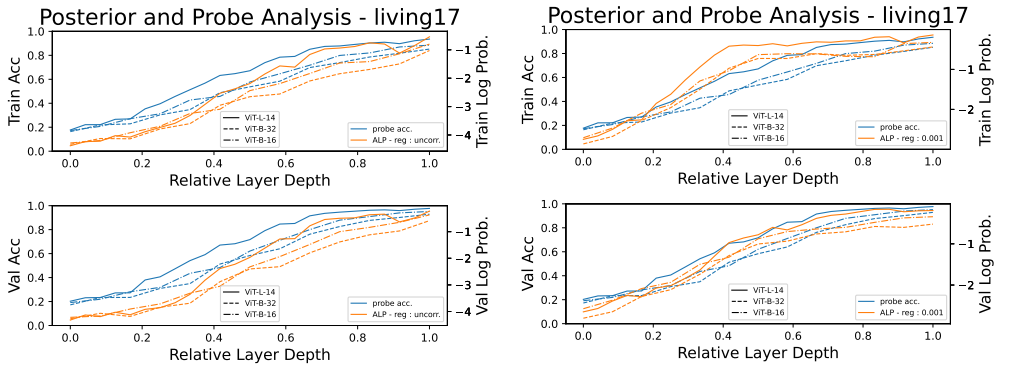
Figure 11: Regularization wise analysis of layerwise ALP vs. linear probe performance for entity 30 breeds dataset.



(a) Regularization 10^{-6}.

(b) Regularization 10^{-9}.

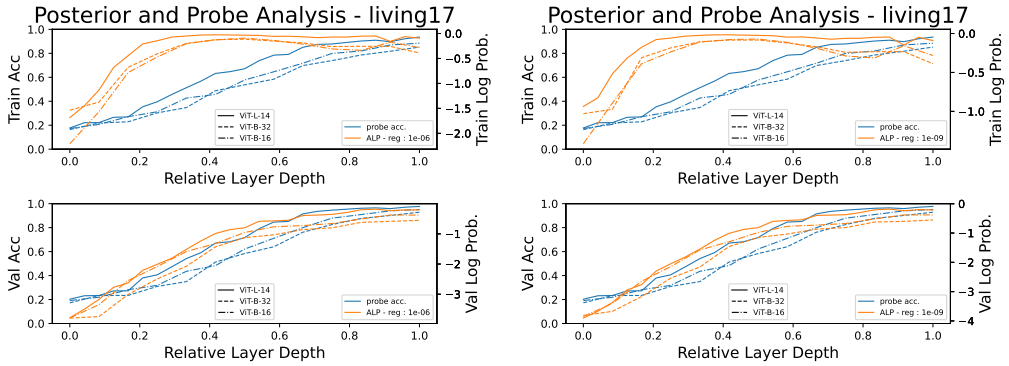
Figure 12: Regularization wise analysis of layerwise ALP vs. linear probe performance for entity 30 breeds dataset.



(a) Regularization - Uncorr.

(b) Regularization 10^{-3}.

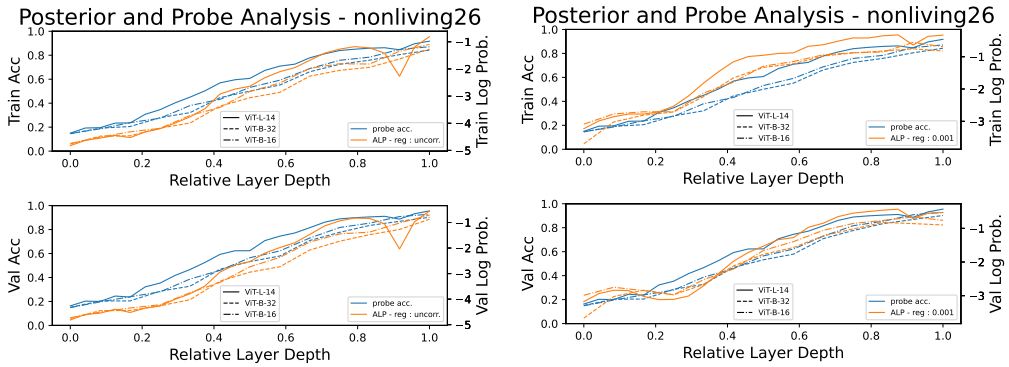
Figure 13: Regularization wise analysis of layerwise ALP vs. linear probe performance for living 17 breeds dataset.



(a) Regularization 10^{-6}.

(b) Regularization 10^{-9}.

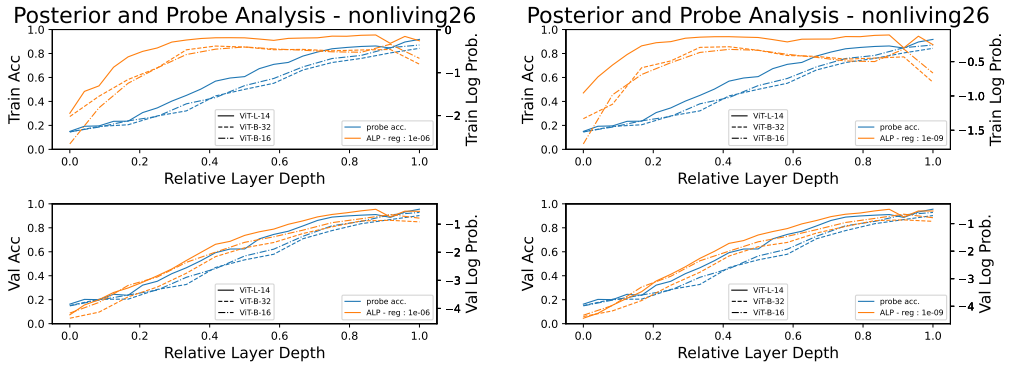
Figure 14: Regularization wise analysis of layerwise ALP vs. linear probe performance for living 17 breeds dataset.



(a) Regularization - Uncorr.

(b) Regularization 10^{-3}.

Figure 15: Regularization-wise analysis of layerwise ALP vs. linear probe performance for nonliving 26 breeds dataset.



(a) Regularization 10^{-6}.

(b) Regularization 10^{-9}.

Figure 16: Regularization-wise analysis of layerwise ALP vs. linear probe performance for nonliving 26 breeds dataset.